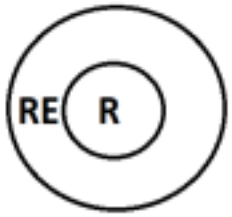
AA - Lab 4

Nu va faceti griji daca nu gasiti transformari pentru reduceri sau nu aveti idei la constructia masinilor Turing din prima incercare, e nevoie sa intelegeti logica din exemplele acestea si apoi sa lucrati voi alte exercitii. E nevoie sa intelegeti bine conceptele de acceptabilitate si decidabilitate. Dar la examen pot aparea subiecte de acest gen.

Clase de probleme



R = multimea problemelor **decidabile**
= {f in $\mathbb{D}$ | f este decisa de o masina Turing}
RE = multimea problemelor **acceptabile**
= {f in $\mathbb{D}$ | f este acceptata de o masina Turing}
R $\subset$ RE

!!! Halting Problem $\in$ RE\R

REDUCERI TURING

Notatia **$f \leq_m g$**
→ g este "mai grea" decat f / f este "mai usoara"
→ f se reduce Turing la g
→ daca avem un algoritm/masina Turing pentu g, avem una si pentru f
$f \leq_m g \Leftrightarrow$ exista o transformare computabilă t(se poate computa cu o masina Turing ca in laborator 1) astfel incat, pentru orice cuvant w pentru care f(w) = TRUE, avem g(t(w)) = TRUE

Ceea ce trebuie sa demonstam: **f(w) = TRUE $\Leftrightarrow$ g(t(w)) = TRUE**
1. construim M_t
2. Aratam ca f(w) = TRUE => g(t(w)) = TRUE
3. Aratam ca g(t(w)) = TRUE => f(w) = TRUE

De retinut!
- **f $\notin$ R => g $\notin$ R**
- **f $\notin$ RE => g $\notin$ RE**
- **g $\in$ R => f $\in$ R**
- **g $\in$ RE => f $\in$ RE**

Observatie:
- O probema poate fi acceptata sau decisa de o MT.
- Rezolvam o problmea = trebui o sa o decidem.

Exercitii

1)
Dați exemplu de o Masina Turing pentru care nu putem determina algoritmic daca se termina sau nu.
(Cu alte cuvinte: nu putem decide daca se termina sau nu)
Raspuns: Masina Turing Universala
MTU[enc(M,w))] → dacă putem decide că MTU[enc(M, w)] se termină, putem decide dacă M[w]

se termină, deci putem decide HALT. Contradicție.

2)

Dati exemplu de o Masina Turing pentru care putem determina algoritmic, pentru orice input, dacă se termină sau nu.

Rezolvare:

Fie masina Alg care primeste o masina M si un cuvant w si determina daca M[w] se termina dupa un numar finit de pasi.

Alg[enc(M, w))]:
for i = 1:50
step(M, w)
dacă M[w] halts
transition to Y
else
transition to N

Astfel, raspunsul la intrebare este o masina M care se termina in maxim 50 pasi.

M[x]: face 50 tranzitii la dreapta si se opreste in Y, N sau H

3)

Determinați exact unde se află următoarele probleme (R, RE, în afară RE):

Rezolvare:

Folosim notatiile cunoscute deja:
step = face o singura tranzitie din starea curenta
run = ruleaza toata masina, pana ori se termina (stare finala sau nu), sau cicleaza

a)

$f_{or}(M_1, M_2) = \text{TRUE} \Leftrightarrow M_1[111] \rightarrow \text{TRUE} \vee M_2[111] \rightarrow \text{TRUE}$

Aratam ca $f_{or} \in RE, \notin R$

Pentru $\in RE$:

M_{or}[enc((M_1, M_2))]:
while(true) {
step(M_1[111])
step(M_2[111])
if(una din ele accepta)
transition to Y
}

Pentru $\notin R$ folosim o Reducere Turing:

Halting Problem $\notin R$

$f_h \leq_m f_{or}$

M, w —t—→ M_1, M_2

Transformarea T:
M1[v]: loop
M2[v]: run M[w]
transition to Y

"=>"

$f_h(M, w) = \text{TRUE}$ => M[w] se termina => M_2 ajunge la final=> $M_2[111) \rightarrow \text{TRUE}$
=> $M_1[111] \rightarrow \text{TRUE} \vee M_2[111] \rightarrow \text{TRUE}$ => $f_{or}(M_1, M_2) = \text{TRUE}$

"<="

$f_{or}(M_1, M_2)$ = TRUE => $M_1[111] \rightarrow$ TRUE $\vee$ $M_2[111] \rightarrow$ TRUE => din transformare, M_1 nu se termina

=> $M_2[111]$ = TRUE => M[w] s-a oprit => $f_h(M, w)$ = TRUE

b)

$f_{and}(M_1, M_2)$ = TRUE $\Leftrightarrow$ $M_1[111] \rightarrow$ TRUE $\wedge$ $M_2[111] \rightarrow$ TRUE

Aratam ca $f_{and} \in$ RE, $\notin$ R

Pentru $\in$ RE:

$M_{and}(M_1, M_2)$:

```
while(true) {
    step(M1[111]) // daca un step a acceptat, la urmatoare iteratie rame pe loc
    step(M2[111])
    if(ambele au acceptat)
        transition to Y
}
```

Pentru $\notin$ R:

$f_h \leq_m f_{and}$

M, w —t—→ M_1, M_2

Transformarea t:

$M_1[x]$: transition to Y

$M_2[x]$: run M[w]

transition to Y

Alternativ: $M_1 = M_2 = M_2$ definit mai sus

"=>"

$f_h(M, w)$ =TRUE => M[w] halts => $M_2[x]$ se termina => $M_2[111] \rightarrow$ TRUE => stim ca si $M_1[111] \rightarrow$ TRUE => $M_1[111] \rightarrow$ TRUE $\wedge$ $M_2[111] \rightarrow$ TRUE => $f_{and}(M_1, M_2)$ = TRUE

"<="

$f_{and}(M_1, M_2)$ = TRUE => $M_1[111] \rightarrow$ TRUE $\wedge$ $M_2[111] \rightarrow$ TRUE

=> Din constructie, $M_2[111]$ = TRUE inseamna ca M[w] se termina => $f_h(M, w)$ = TRUE

c)

$f_{diff}(M_1, M_2)$ = TRUE $\Leftrightarrow$ $M_1[111] \rightarrow$ TRUE $\wedge$ $M_2[111]$ –/→ TRUE

Situatiile posibile:

$M_1[111] \rightarrow$ FALSE(rejects) => f_{diff} = 0

$M_1[111] \rightarrow$ FALSE (accepts) =>

| $M_2[111] \rightarrow$ FALSE => $f_{diff}(M_1, M_2)$ = TRUE

| $M_2[111]$ loop => $f_{diff}(M_1, M_2)$ = TRUE

| $M_2[111] \rightarrow$ TRUE=> $f_{diff}(M_1, M_2)$ = FALSE

→ nu putem decide daca o masina se termina sau nu

=> ar trebui sa fie cel mult acceptabila ($\in$ RE, $\notin$ R)

→ Incercam sa vedem daca $\in$ RE: construim masina care accepta problma

$M_{diff}[M_1, M_2]$:

```
while(true) {
    step(M1[111])
    step(M2[111])
```

if(M_1 accepta) {....}

// !!! nu putem verifca daca M_2respinge sau doar cicleaza

}

=> s-ar putea ca problema sa nu fie nici acceptabila

→ Aratam ca $\notin$ RE cu o reducere Turing la Looping Problem

$f_l \notin$ RE

fl(M,w) = TRUE, daca M[w] loops

FALSE daca se opreste

$f_l \leq_m f_{diff}$

M, w —t—→ M_1, M_2

Transformarea T:

M_1[x]: transition to Y

M_2[x]: run M[w]

transition to Y

"=>"

f_l(M, w) = TRUE => M[w] cicleaza => M_2[111] cicleaza => M_2[111] –/→ TRUE => din transformare, M_1[111]→TRUE $\wedge$ M_2[111] –/→ TRUE => $f_{diff}(M_1, M_2)$ = TRUE

"<="

$f_{diff}(M_1, M_2)$ = TRUE => M_1[111]→TRUE $\wedge$ M_2[111] –/→ TRUE

=> Asa cum am construit M_2, M_2[111] –/→ TRUE inseamna ca M_2[111] cicleaza (nu respinge niciodată, conform constructiei) => M[w] cicleaza => f_l(M, w) = TRUE

4)

Pentru fiecare dintre problemele urmatoare, determinati daca sunt decidabile sau nu (demonstratia nedecidablitatii poate fi realizata printr-o reducere Turing, cea a decidabilitatii prin schitarea unui algoritm):

a)

f_s(M) = TRUE ⇔ ∀w, M[w] nu muta niciodata capul de citire la stanga

este decidabila?

Problema primeste enc(M) ca input: M_s[enc(M)]

→ cauta toate tranzittile la stanga (r, c_1) → (s, c_2, →)

→ ne vom asigura ca nu putem ajunge la (r,c_1) din q_1 => (imaginati-va tranzitiile ca un graf orientat; simbolul citit este eticheta muchiei. Dacă nu există nicio cale de la q_1 către o stare care are pe vreun simbol o tranziție cu mișcare la stânga, mașina nu va muta niciodată capul la stânga.

Dacă există o cale către o stare care are definită o tranziție la stânga pe un simbol din alfabetul de input, sau pe simbolul blanc, atunci există cel puțin un input pentru care mașina mută capul la stânga.

Cazul mai complicat e în care putem ajunge la o stare care are definită o tranziție cu mutarea capului spre stânga dacă citește un simbol non-blanc care nu este în alfabetul de input. Trebuie să verificăm dacă acel simbol poate fi scris dintr-o stare anterioară, care păstrează capul nemișcat.)

=> problema este decidabila

b)

f_{1010}(M) = TRUE ⇔ M[1010] nu muta niciodată capul de citire la stânga

Daca incercam o abordare obisnuita precum

while(...) // doar vede acceptabilitatea

step(M[1010])

nu am prea putea verifica propriu-zis daca este decidabila sau nu

Asa ca trebuie sa verificam decidabilitatea diferit:

→ Care ar fi numarul max de pasi pentru care ne-am asigura ca M nu va face sigur niciun pas la stanga?

→ Cand putem face pasi la stanga:

- poate fi un la stanga exact la inceput din q_1
- putem face un pas la stanga in timp ce parcurgem spre dreapta inputul - 1010#########

daca nu se termina, masina cicleaza

– la finalul inputului (q' ,#) → (q', #, R/H)

sau

– pe parcusul inputlui (q' ,0/1) → (q', 0/1, H)

Deci, putem avea un comportament similar cu cel prezentat mai jos pana cand ajungem la un ciclu din unul din cazurile de mai sus. Luam primul caz mentionat:

Se fac tranzitii la dreapta/hold pana se ajung la finalul cuvantului.

Apoi tranzitiile arata in felul urmator:

(q', #) → (q'', #, R/H) → (q''', #, R/H) → ... de |K| ori

Daca avem |K| stari, se pot face maxim |K| tranzitii la dreapata schimband starile. Deci, numarul de pasi pentru a verifica ca M[1010] nu muta cursorul la stanga este: (1 + lungime input + |K|) pasi

M_{1010}[M]:

k = 0

while(k < 1 + 4 + |K|) { // inputul 1010 are lungimea 4

step(M[1010])

if(tranzitia nu a fost la stanga)

k++

else

transition to N

}

transition to Y

Problema este decidabila.

c)

f_{50a}(M) = TRUE ⇔ M[1010] nu trece prin mai mult de 50 de tranziții

- 1 tranzitie = 1 step
- rulam 50 de pasi din M[1010]

→ daca se termina intre timp => f_{50a}(M) = TRUE

→ daca nu => trecem prin > 50 tranzitii => f_{50a}(M) = FALSE

=> decidabila

d)

f_{50b}(M) = TRUE ⇔ ∀w, M[w] nu trece prin mai mult de 50 de tranziții

- dacă mașina citește tot inputul, atunci face minim |w| tranziții
- deci singura situație în care ar face mai puțin de 50 de tranziții e dacă nu încearcă să citească tot inputul, ci doar maxim 51 de simboluri

- e suficient să rulăm pe toate inputurile w cu |w| <= 51, pentru maxim 50 de tranziții (sau până se oprește)
- dacă s-a oprit pentru toate, atunci se oprește pentru orice prefix de lungime 51, nu mai e interesată de restul inputului, deci nu contează cu cât îl lungim → transition to Y
- (trebuie să rulăm și pentru cuvintele cu |w| < 51 pentru că poate ajunge la blanc de la sfârșit și să aibă comportament diferit decât pentru simboluri de input) ● dacă pe unul din cuvintele astea nu s-a terminat, înseamnă că am oprit simularea când urma să facă tranziția #51 → transition to N

=> decidabila